



Higher Diploma in Computer Science

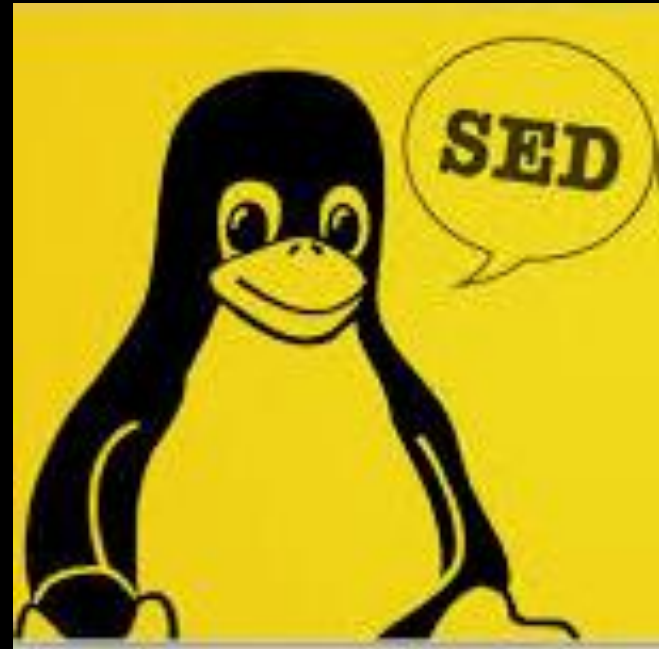
Computer Systems & Networks



Waterford Institute of Technology
INSTITIÚID TEICNEOLAÍOCHTA PHORT LAIRGE

sed

for finding, replacing,
deleting, and inserting text.



sed

- stream editor

sed is a special editor for **modifying files** automatically

- make global changes to a large file
- Allows a script make changes to a file

sed requires **an input file** and a **command** that lets sed know what actions to apply to the file

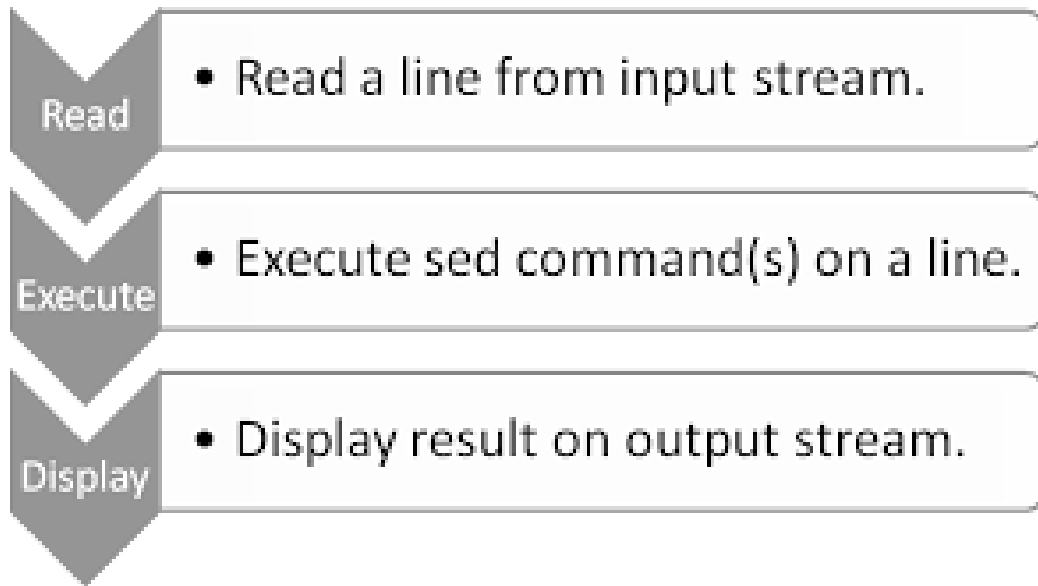
sed uses:

- Text substitution
- Selective printing of text files
- Replace text in-place (modify the file)
- Non-interactive editing of text files etc.

⚠ *irreversible unless backed up!!*

sed

- stream editor



sed can be used:

- at the command-line, or
- within a shell script to edit a file non-interactively

= *no need to open a text editor*
(ie I could replace all occurrences of 2024 with 2025)

Syntax

\$sed '/pattern/ command' my_file

Where ***pattern*** is a regex and
command can be one of:

s = search & replace, or

p = print, or

d = delete, or

i=insert, or

a=append etc.

Substitute Text (s)

\$ sed 's/cat/dog/'

The diagram shows the command 's/cat/dog/' with labels: 's' is the command, '/' is the delimiter, 'cat' is the pattern, and 'dog' is the replacement.

\$sed 's/input/output/' my_file

echo every line from **my_file** to STDOUT, changing **the first occurrence** of the string 'input' on each line into 'output'

= typically used to replace a word

NOTE: **sed** is line-oriented

- replaces the first occurrence & moves to next line
- if you wish to change **every occurrence** on each line, then you need to make it a 'greedy' global search & replace like so:

\$sed -e 's/input/output/g' my_file

Remove (Delete) Lines (**d**)

\$sed '/pattern/d' my_file

echo every line from *my_file* to STDOUT, deleting lines matching the pattern, for example:

\$sed '/^\$/d' file.txt

Replace only on line 2:

\$sed '2s/foo/bar/' file.txt

Print only Matching Lines (**p**)

\$sed -n '/pattern/p' my_file

- n:** suppress normal output (by default, sed prints every line)
- /pattern/** matches lines containing the given pattern.
- p** print only matching lines

Case-Insensitive Matching (**I**)

\$sed -n '/pattern/Ip' my_file

- I** (Captial i) makes the match case-insensitive

Escape Symbol

Replace **/bin** with **/usr/local/bin** in a file called **my_script**, and write the result to a new file called **new_script**:

```
sed -e 's/\bin/\usr/local/bin/' my_script > new_script
```



AHH, it's so confusing looking!

Any single character can be used as the delimiter in the **s///** command

- instead of **/**, you can use **#**, **@**, **|**, **:**, etc., as long as you're consistent

```
$sed 's#/bin#/usr/local/bin#' my_script > new_script
```

- cleaner and easier to read

Wildcards

- You want to take every line that starts with a number in your file and surround that number by brackets (try this with your **employee.txt** file)

```
sed -e 's/[0-9]*/(&)/' my_file
```

Redirecting output

- By default the output is written to STDOUT.

```
sed -e 's/input/output/' my_file > new_file
```

- e** used to **run multiple sed commands** in a single call
is optional when running ***only one*** command

```
sed -i -e 's/input/output/' my_file
```

- i** Edits my_file directly, replacing all occurrences of “input” with “output”

Examples to try out

- Print out only directories (i.e. they begin with a “d” in `ls -l`)

```
$ls -l | sed -n -e '/^d/ p'
```

- You wanted to delete all lines that start with the comment symbol '#'
you could use

```
$sed -e '/^#/ d' my_file
```

Remove Deleted Lines (**d**)

- You can also use the special line number '\$' to mean 'end of file'.
 - if you wanted to delete all but the first 10 lines of a file:

```
$sed -e '11,$ d' my_file
```

- You can also use a **pattern-range** form, where the first regexp defines the start of the range, and the second the stop:

```
$sed -n -e '/caroline/,/rosanne/p' employee.txt
```

Try out this weeks Lab

03: if, while, for, break & continue; case & select for menus; using sed editor

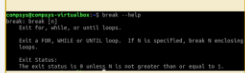
01: Conditionals and Loops



LOOPS REPEAT ACTIONS... SO YOU DON'T HAVE TO

test · if · for · while · until

02: Loop manipulation



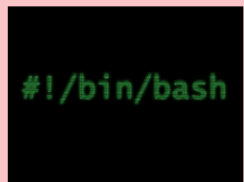
break · continue · case · select

03: sed



Stream editor · find and replace

Lab



Loops · Menus · sed

Computer Systems > 03: Loops, Menus > Week 3 Class 1... > Lab

Advancing Scripting Skills

01. Conditionals

02. Loops

03. Menu driven scripts

04. sed

05. EXERCISE Add new records

06. Cisco NetAcad

04. sed

sed editor · find and replace

For this lab, download [Distros.txt](#) file, a space separated text file, and try the following commands:

- Print lines 7-9 inclusive:

```
$ sed -n '7,9p' distros.txt
```

- Search for a pattern and print it:

```
$ sed -n '/SUSE/p' distros.txt # print lines matched by "SUSE"
```

- Negate the above search pattern:

```
$ sed -n '/SUSE/lp' distros.txt #except the ones matched by X
```

PRACTICE using sed

Make a copy of the **distros.txt** file, named **tempdistros.txt** and practice deleting sections of the file

```
$ sed '1d' tempdistros.txt #deletes 1st line
```

Try out this weeks Lab

T > Home > Computer Syste... > 03: Loops, Men... > Week 3 Class 1... > Lab

Advancing Scripting Skills

01. Conditionals

02. Loops

03. Menu driven scripts

04. sed

05. EXER Add new records

06. Cisco NetAcad

05. EXER Add new records

EXERCISE

Write a script that can be used to add a new record (employees details) to your **employee.txt** file

TIPS

Think about this logically:

- 1st check if the file exists and maybe do you want to create it if it's not there? Do you want to use to receive error messages Customised feedback?
- Then you want to give appropriate variable names in order to the positional parameter, the 1st parameter being the employee's ID, the second their name, the third their position in the company. 4th the dept they belong to and finally, 5th the expenses they claimed.
- Whats next? Time to take the details in and append them to your **employee.txt** file

ERROR CHECKING:

Error checking is an **absolute MUST** especially when you're taking user input into a system to ensure that the expected input is in the expected format i.e. email addresses, phone numbers, postcodes etc. are all in the expected format

Advancing Scripting Skills

01. Conditionals

02. Loops

03. Menu driven scripts

04. sed

05. EXER Add new records

06. Cisco NetAcad

06. Cisco NetAcad

CA · Chapter Exams

For the purpose of this module, we will today complete the final two chapters and their associated Chapter Exams.

- For the 5% CA, you are required to have completed the chapter exams from Chapter 1 up to and including Chapter 11



5% Cisco
NetAcad
chapter exams

Final NetAcad 5% CA

Completed Chapter Exams *up to and including* Chapter 11

